

Resistance Training using Prior Bias: toward Unbiased Scene Graph Generation

Chao Chen^{1,2*}, Yibing Zhan², Baosheng Yu³, Liu Liu³, Yong Luo^{1†}, Bo Du^{1†}

¹ National Engineering Research Center for Multimedia Software, Institute of Artificial Intelligence, School of Computer Science and Hubei Key Laboratory of Multimedia and Network Communication Engineering, Wuhan University, China,

² JD Explore Academy, China, ³ The University of Sydney, Australia,

{chenchao, luoyong, dubo}@whu.edu.cn,

zhanyibing@jd.com, baosheng.yu@sydney.edu.au, liu.liu1@sydney.edu.au,

Abstract

Scene Graph Generation (SGG) aims to build a structured representation of a scene using objects and pairwise relationships, which benefits downstream tasks. However, current SGG methods usually suffer from sub-optimal scene graph generation because of the long-tailed distribution of training data. To address this problem, we propose Resistance Training using Prior Bias (RTPB) for the scene graph generation. Specifically, RTPB uses a distributed-based prior bias to improve models' detecting ability on less frequent relationships during training, thus improving the model generalizability on tail categories. In addition, to further explore the contextual information of objects and relationships, we design a contextual encoding backbone network, termed as Dual Transformer (DTrans). We perform extensive experiments on a very popular benchmark, VG150, to demonstrate the effectiveness of our method for the unbiased scene graph generation. In specific, our RTPB achieves an improvement of over 10% under the mean recall when applied to current SGG methods. Furthermore, DTrans with RTPB outperforms nearly all state-of-the-art methods with a large margin.¹

1 Introduction

Scene graph generation aims to understand the semantic content of an image via a scene graph, where nodes indicate visual objects and edges indicate pairwise object relationships. An intuitive example of scene graph generation is shown in Figure 1. Scene graph generation is beneficial to bridge the gap between the low-level visual perceiving data and the high-level semantic description. Therefore, a reliable scene graph can provide powerful support for downstream tasks, such as image captioning (?), image retrieval (?), and visual question answering (?).

Scene graph generation usually suffers from the long-tail problem of relationships in training data(???). For example, as shown in Figure 1(c), the widely used scene graph generation dataset, Visual Genome (?), is dominated by a few relationships with coarse-grained descriptions (head categories), whereas there are no sufficient annotations available

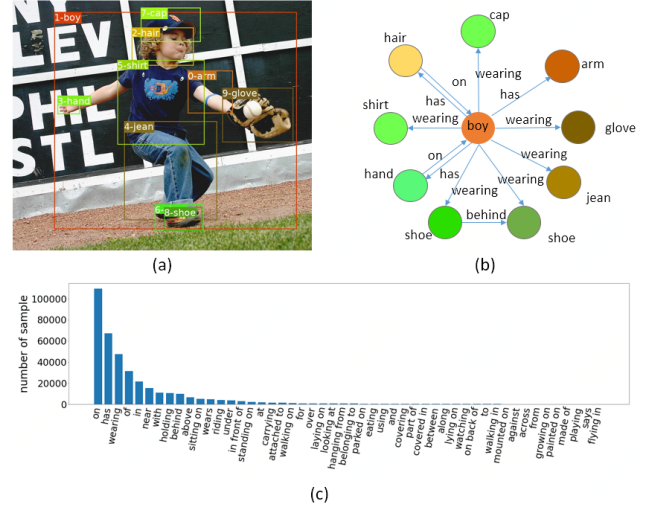


Figure 1: (a) an input image with detected objects. (b) a generated scene graph, which is a graphical representation of the input image with objects and pairwise relationships. (c) the long-tail distribution of object relationships in the Visual Genome (VG) dataset (?).

for other less frequent relationships (tail categories). This severe imbalanced distribution makes it difficult for an unbiased scene graph generation, considering there are only a few training samples in real-world scenarios for recognizing less frequent relationships.

To address the above-mentioned issues, many class rebalancing strategies have been introduced for unbiased scene graph generation(?). However, existing methods still struggle to achieve satisfactory performance on tail categories, and more sophisticated solutions are desirable. Inspired that humans increase muscle strength by making their muscles work against a force, we propose resistance training using prior bias (RTPB) to improve the detection of less-frequently relationships and address the long-tail problem in SGG. Specifically, RTPB assigns a class-specific resistance to the model during training, where the resistance on each relationship is determined by a prior bias, named resistance bias. The resistance bias is first initialized by the prior statistic about the relationships in the training set. The motivation

*This work was done when Chao Chen was a research intern at JD Explore Academy.

†Corresponding Author.

Copyright © 2022, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

¹Code will be available at <https://github.com/ChCh1999/RTPB>

of a resistance bias is to enforce the model to strengthen the ability on less frequent relationships, since they are always corresponding to heavier resistance. In this way, RTPB enables the model to resist the influence of the imbalanced training dataset. Furthermore, to better explore global features for recognizing the relationships, we design a contextual encoding backbone network, named dual Transformer (DTrans for short). Specifically, the DTrans uses two stacks of the Transformer to encode the global information for objects and relationships sequentially. To evaluate the effectiveness of the proposed RTPB, we introduce it into recent state-of-the-art methods and our DTrans. Specifically, RTPB can bring an improvement of over 10% in terms of the mean recall criterion compared with other methods. By integrating with the DTrans baseline, RTPB achieves a new state-of-the-art performance for the unbiased scene graph generation.

The main contributions of this paper are summarized as follows: 1) We propose a novel resistance training strategy using prior bias (RTPB) to improve the model generalizability on the rare relationships in the training dataset; 2) We define a general form of resistance bias and devise different types of specific bias by using different ways to model object relationship; 3) We introduce a contextual encoding structure based on the transformer to form a simple yet effective baseline for scene graph generation. Extensive experiments on a very popular benchmark demonstrate significant improvements of our RTPB and DTrans compared with many competitive counterparts for unbiased SGG.

2 Related Work

2.1 Scene Graph Generation

Scene graph generation belongs to visual relationship detection and uses a graph to represent visual objects and the relationships between them (?). In early works, detection and prediction are performed only based on specific objects or object pairs (?), which can't make full use of the semantic information in the input image. To take the contextual information into consideration, later works utilize the whole image by designing various network architectures, such as biLSTM (?), TreeLSTM (?) and GNN (??). Some other works try to utilize external information, such as linguistic knowledge and knowledge graph, to further improve scene graph generation (????).

However, due to the annotator preference and image distribution, this task suffers from a severe data imbalance issue. Dozens of works thus try to address this issue, and the main focus is on reducing the influence of long-tail distribution and building a balanced model for relationship prediction. Tang et al. also try to use causal analysis (?) to reduce the influence of training data distribution on the final model. Some other works (???) address this issue in a positive-unlabeled learning manner, and typical imbalance learning methods, such as re-sampling and cost-sensitive learning, are also introduced for scene graph generation (??). Unlike these approaches, we adopt the resistance training strategy using prior bias (RTPB), which utilizes a resistance bias item for the relationship classifier during training to optimize the loss value and the classification margin of each type of rela-

tionship.

2.2 Imbalanced Learning

Imbalanced learning methods can be roughly divided into two categories: resampling and cost-sensitive learning.

Resampling Resampling methods change the class ratio to make the dataset a balanced one. These methods can be divided into two categories: oversampling and undersampling. Oversampling increases the count of the less frequent classes (?), and hence may lead to overfitting for minority classes. Undersampling reduces the sample of major categories, and thus is not feasible when data imbalance is severe since it discards a portion of valuable data (?).

Cost-sensitive Learning Cost-sensitive learning assigns different weights to samples of different categories (?). Reweighting is a widely used cost-sensitive strategy by using prior knowledge to balance the weights across categories. Early works use the reciprocal of their frequency or a smoothed version of the inverse square root of class frequency to adjust the training loss of different classes. However, this tends to increase the difficulty of model optimization under extreme data imbalanced settings and large-scale scenarios. ? propose class balanced (CB) weight, which is defined as the inverse effective number of samples(?). Some other approaches dynamically adjust the weight for each sample based on the model's performance (???).

Different from these approaches, which directly deal with the loss function, we add a prior bias on the classification logits for each class based on the distribution of training dataset. In this way, we provide a new approach to import additional category-specific information and address the imbalance problem through adjusting the classification boundaries. A similar idea is proposed in (?) for long-tailed recognition, and we differ from them significantly in the tasks and formulations.

3 Methods

3.1 Problem Setting and Overview

Problem Setting Given an image $\mathcal{I}$, our goal is to predicate a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where $\mathcal{V}$ is the set of objects in the image and $\mathcal{E}$ is pairwise relationships of objects in $\mathcal{V}$. Each object $v \in \mathcal{V}$ consists of the bounding box coordinates $\mathbf{b}_v$ for location and the class label c_v for type. An edge $e \in \mathcal{E}$ should include the pair of a subject and a object, i.e., (v_s, v_o) , and the label of the relationship r between v_s and v_o . Typically, for object $v \in \mathcal{V}$, the label c_v belongs to $\mathcal{C}_e = \{1, 2, \dots, L_e\}$, where L_e is the number of object labels. The label of relationship c_r are in $\mathcal{C}_r = \{1, 2, \dots, L_r\}$, where L_r is the number of relationship labels.

In the commonly used SGG pipeline, the probability of scene graph $Pr(\mathcal{G}|\mathcal{I})$ is formulated as follows

$$Pr(\mathcal{G}|\mathcal{I}) = Pr(\mathcal{B}|\mathcal{I})Pr(\mathcal{O}|\mathcal{I}, \mathcal{B})Pr(\mathcal{G}|\mathcal{I}, \mathcal{B}, \mathcal{O}), \quad (1)$$

where $Pr(\mathcal{B}|\mathcal{I})$ indicates the proposal generation. The $\mathcal{B}$ is the set of bounding box $\mathbf{b}_v$, which is conducted by an object detector. $Pr(\mathcal{O}|\mathcal{I}, \mathcal{B})$ denotes the object classification. The $\mathcal{O}$ is the set of object class c_v , and $Pr(\mathcal{G}|\mathcal{I}, \mathcal{B}, \mathcal{O})$ means the final relationship classification.

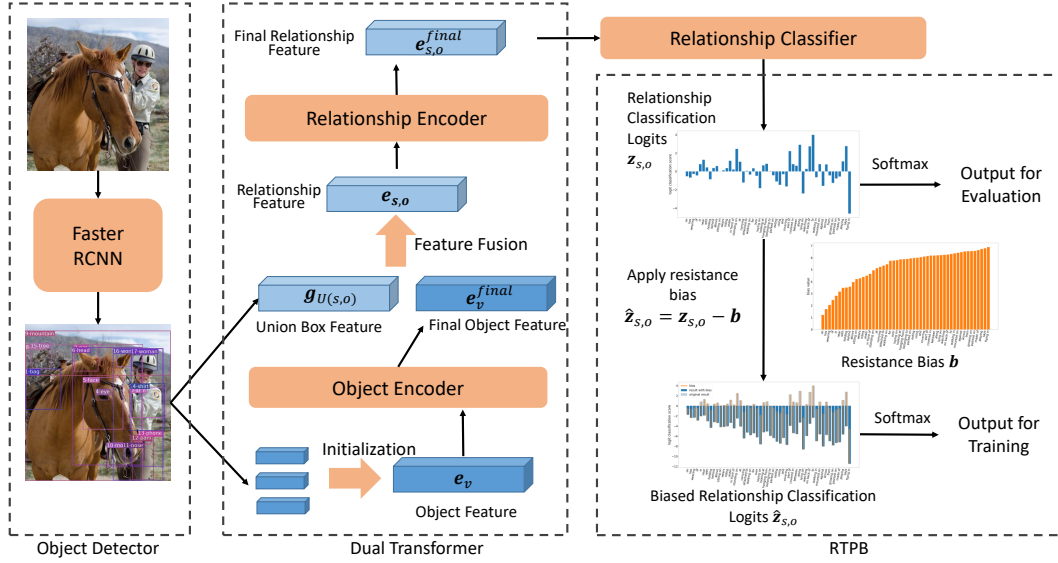


Figure 2: The overall structure of our method. We adopt Faster RCNN as the object detector. Dual Transformer is used to encode object/relationship features. RTPB addresses the long-tail problem by adjusting the classification logits of each relationship.

Methods Overview Figure 2 shows the overall structure of our method. Following previous works (??), we utilize Faster RCNN (?) to generate object proposals and the corresponding features from the input image. For each object proposal, the object detector conducts the bounding box coordinates $\mathbf{b}_v$, the visual feature $\mathbf{g}_v$, and the object classification score $\mathbf{z}_v \in \mathbb{R}^{|C_v|}$. Besides, to improve the relationship prediction, the backbone also generates the feature of the union boxes of each pair of objects. Then, we introduce our *Dual Transformer (DTrans)*, which uses two stacks of Transformers to encode the context-aware representations for objects and relationships, respectively (see Section 3.2). Moreover, to address the long-tail problem for SGG, we adopt the *resistance training using prior bias (RTPB)* for unbiased SGG. The RTPB assigns resistance to the model by the prior bias, named resistance bias (see Section 3.3).

3.2 Dual Transformer

We propose the Dual Transformer (DTrans) for better contextual information encoding. As shown in Figure 2, based on the outputs of the object detector, i.e., the location of objects, the feature of objects, and the feature of union boxes, the DTrans uses self-attention to encode the context-aware representation of objects and relationships.

Self-attention Our model uses two stacks of transformers to obtain the contextual information for objects and the corresponding pairwise relationships. Each transformer encodes the input features with self-attention mechanisms (?), which mainly consists of the attention and the feed-forward network. The attention matrix is calculated as

$$\text{Attention}(Q, K, V) = \sigma\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (2)$$

where query (Q), keys (K), and value (V) are obtained from the input feature through three different linear transforma-

tion layers, $1/\sqrt{d_k}$ is the scaling factor for the dot product of Q and K (?), and σ is the softmax function. Besides, we utilize the multi-head attention, which is used to divide the Q , K , and V into n_h parts and calculates the attentions of each part. Next, a feed-forward network (FFN) is used to mix the attentions of each part up.

Object Encoder As shown in Figure 2, based on all the results about object proposals from the object detector, we first compose them to initialize a feature vector of the corresponding object as follows:

$$\mathbf{e}_v = W_o[\text{pos}(\mathbf{b}_v), \mathbf{g}_v, \text{ebd}(c_v)], \quad (3)$$

where $[\cdot, \cdot]$ denotes the concatenation operation, the pos is a learnable position encoding method for object location, $\text{ebd}(c_v)$ is the GloVe vector embedding for object label c_v , and W_o is a linear transformation layer used to initialize the object feature from the above information. Then, we feed the $\mathbf{e}_v$ into a stack of n_o transformers to obtain the final context-aware features $\mathbf{e}_v^{\text{final}}$ for objects. The $\mathbf{e}_v^{\text{final}}$ is first used to produce the final classification result of objects as follows:

$$\mathbf{p}_v = \sigma(W_{clf}^o \mathbf{e}_v^{\text{final}}), \quad (4)$$

where W_{clf}^o is the object classifier. And then, the $\mathbf{e}_v^{\text{final}}$ is also used to generate the features of the pairwise relationships of objects.

Relationship Encoder Before encoding the features of relationships, we first carry out a feature fusion operation to generate the basic representation of relationships. For each directed object pair (s, o) , we concatenate the visual feature $\mathbf{g}_{U(s,o)}$ of the union box of s and o , and the context-aware features of the subject s and the object o . Then, we use a linear transformation to obtain the representation of the relationship between the subject s and the object o :

$$\mathbf{e}_{s,o} = W_r[\mathbf{g}_{U(s,o)}, \mathbf{e}_s^{\text{final}}, \mathbf{e}_o^{\text{final}}], \quad (5)$$

where W_r is a linear transformation layer used to compress the relationship features. $\mathbf{e}_s^{final}$ and $\mathbf{e}_o^{final}$ are the final context-aware features of subject s and object o .

Then, we use another stack of n_r Transformers to encode the features of relationships and produce the final feature $\mathbf{e}_{s,o}^{final}$ for the relationship between subject s and object o . Finally, the relationship classifier uses the feature $\mathbf{e}_{s,o}^{final}$ to recognize the pairwise relationships as follows

$$\mathbf{p}_{s,o} = \sigma(\mathbf{z}_{s,o}) = \sigma(W_{cls}^r \mathbf{e}_{s,o}^{final}), \quad (6)$$

where $\mathbf{z}_{s,o} = W_{cls} \mathbf{e}_{s,o}^{final}$ is vector of the logits for relationship classification, and W_{cls}^r is the relationship classifier.

3.3 Resistance Training using Prior Bias

Inspired by the idea of resistance training that muscle turns to be stronger if they are training with heavier resistance (?), we propose the resistance training using prior bias (RTPB) for unbiased SGG. Our RTPB uses the prior bias as the resistance for the model only during training to adjust the model's strength for different relationships. We name the prior bias as resistance bias, which is applied to the model in the training phase as:

$$\hat{\mathbf{z}}_{s,o} = \mathbf{z}_{s,o} - \mathbf{b}, \quad (7)$$

where $\mathbf{z}_{s,o} = [z_1, z_2, \dots, z_{L_r}]$ is the vector of logits for relationship classification in Eq. (6), and $\mathbf{b} = [b_1, b_2, \dots, b_{L_r}]$ is the vector of resistance bias for each relationship. By assigning relatively heavier resistance to the tail relationships during training, we enable the model without resistance to handle the tail relationships better and obtain a balanced performance between the tail relationships and the head relationships.

In the following part of this section, we first introduce four resistance biases from the prior statistic about the training set. Then, we analyze how RTPB works through the loss and the objective.

Instances for Resistance Bias We define the basic form of resistance bias b_i for relationship $i \in \mathcal{C}_r$ as follows:

$$b_i = -\log \left(\frac{w_i^a}{\sum_{j \in \mathcal{C}_r} w_j^a} + \epsilon \right), \quad (8)$$

where w_i is the weight of relationship i , and a, ϵ are the hyper-parameters used to adjust the distribution of b_i . For resistance bias, the category weight w_i is negatively correlated to the resistance bias value b_i . Therefore, to assign the head categories of relationship small resistance, the distribution of the weight w_i should be correlated to the long-tail distribution of the training set. We introduce four resistance biases:

Count Resistance Bias (CB) Intuitively, for the general classification task, we can set the bias weight w_i of resistance bias with the proportion of sample for each relationship in the training set. We denote this type of resistance bias as count resistance bias (CB).

Valid Resistance Bias (VB) Besides the number of samples, the long-tail problem also occurs between the general descriptions and the detailed ones. Therefore, we formulate

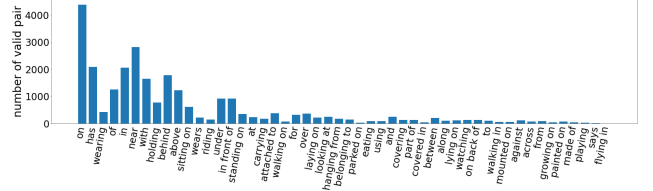


Figure 3: The distribution of the valid pair count.

valid pair as follows: if there is relationship $i \in \mathcal{C}_r$ for object pair (s, o) in the training set, the (s, o) is a valid pair of relationship i . The distribution of valid pair count for the popular SGG data set, Visual Genome (?), is shown in the Figure.3. We set the weight w_i as the proportion of valid object pair for relationship i . In this way, the RTPB can distinguish between the general descriptions of relationships and the rare ones. We call this resistance bias **Valid Resistance Bias (VB)** for short.

Pair Resistance Bias (PB) However, for the SGG task, the classification of relationship is related to not only the relationship, but the pair of entities. Thus we try to use the relationship distribution of each object pair. For different object pairs, the bias are different. In application, we select the bias based on the classification result of objects. For object pair (s, o) , the resistance bias for relationship i is follows:

$$b_{s,o,i} = -\log \left(\frac{w_{s,o,i}^a}{\sum_{j \in \mathcal{C}_r} w_{s,o,j}^a} + \epsilon \right), \quad (9)$$

where $w_{s,o,i}$ is the ratio of the relationship i in all the relationships between subject s and object o in the training set. And we call this resistance bias as **Pair Resistance Bias (PB)**.

Estimated Resistance Bias (EB) Because many pairs of s, o have only a little number of valid relationships and few annotations, the number of relationship sample for particular object pair can hardly show the general distribution of relationship in many cases. Thus we propose subject-predicate and predicate-object count n^{sppo} to estimate the relationship distribution. For subject s , object o and relationship i , it's calculated as

$$n_{s,o,i}^{sppo} = \sqrt{\sum_{o' \in \mathcal{C}_e} n_{s,o',i} \times \sum_{s' \in \mathcal{C}_e} n_{s',o,i}}, \quad (10)$$

where $n_{s,o,i}$ is the count of relationship i between subject s and object o . Then we produce a new resistance bias by replacing the weight $w_{s,o,i}$ in Eq.(9) with the proportion $n_{s,o,i}^{sppo} / \sum_{j \in \mathcal{C}_r} n_{s,o,j}^{sppo}$. We name this type of resistance as **Estimated Resistance Bias (EB)**.

Loss with RTPB In this section, we analyze the effect of RTPB through classification loss. In the classifier, we conduct the final relationship classification probability by a softmax function, and we use the cross-entropy (CE) to evaluate the classification optimization objective. The classification

probability and CE loss are shown in follows,

$$p_i = Pr(i|s, o, \mathcal{I}) = \sigma(\mathbf{z}_{s,o})_i = \frac{e^{z_i}}{\sum_{j \in \mathcal{C}_r} e^{z_j}}, \quad (11)$$

$$\mathcal{L}_i = -\log(p_i) = -\log \frac{e^{z_i}}{\sum_{j \in \mathcal{C}_r} e^{z_j}}, \quad (12)$$

where p_i is the probability that the relationship between object pair (s, o) in image $\mathcal{I}$ is i , and $\mathbf{z}_{s,o}$ is the vector of logits for relationship classification mentioned in Eq. (6). If we apply our RTPB, the probability p_i in Eq. (11) turns to be:

$$\hat{p}_i = \sigma(\mathbf{z}_{s,o} - \mathbf{b})_i = p_i \frac{e^{-b_i}}{\sum_{j \in \mathcal{C}_r} e^{-b_j} p_j},$$

where $\mathbf{b} = [b_1, b_2, \dots, b_{L_r}]$ is vector of resistance bias. And the softmax cross-entropy loss turns to be:

$$\begin{aligned} \hat{\mathcal{L}}_i &= -\log(\hat{p}_i) = -\log \frac{e^{z_i - b_i}}{\sum_{j \in \mathcal{C}_r} e^{z_j - b_j}} \\ &= -\log \frac{e^{z_i}}{\sum_{j \in \mathcal{C}_r} e^{z_j}} + b_i + \log \sum_{j \in \mathcal{C}_r} \frac{e^{-b_j} e^{z_j}}{e^{z_j}} \\ &= \mathcal{L}_i + \theta_i, \end{aligned} \quad (13)$$

where $\theta_i = b_i + \log \sum_{j \in \mathcal{C}_r} e^{-b_j} p_j$. For the loss function, RTPB can be regard as a dynamic re-weighting method based on the resistance bias b_i and the relationship prediction p_i . And the weight for the relationship i between (s, o) is θ_i in Eq. (13). Because $\frac{d\theta_i}{db_i} = 1 - \frac{e^{-b_i} Pr(i|s,o,\mathcal{I})}{\sum_{j \in \mathcal{C}_r} e^{-b_j} Pr(j|s,o,\mathcal{I})} > 0$, the RTPB can up-weight the relationships that correspond to larger resistance bias while down-weighting the rest. In this manner, RTPB reduces the loss contribution from head relationships and highlight the tail relationships to keep a balance between head and tail. Take binary classification as an example, and we can visualize the loss value for the pair with a specific resistance bias value; the figure is shown in the Appendix.

Moreover, the loss value tends to be larger when the predicted distribution of p_i in Eq. (11) is close to the distribution of $e^{-b_i} = \frac{w_i^a}{\sum_{j \in \mathcal{C}_r} w_j^a} + \epsilon$, which is correlated to the distribution of the predefined weight w_i of each relationship in Eq. (8). Because the distribution of w_i correlates to the long-tail distribution of the training set, the resistance bias provides continuous supervision against the long-tailed distribution to avoid biased prediction.

Objective with RTPB From another point of view, we can simply regard the $\hat{\mathbf{z}}_{s,o} = \mathbf{z}_{s,o} - \mathbf{b}$ as a joint to be optimized in the training phase and the result should be close to the baseline without RTPB. We then get the final prediction $\mathbf{z}_{s,o} = \hat{\mathbf{z}}_{s,o} + \mathbf{b}$. In this manner, compared with the model without RTPB, we tend to judge the ambiguous predictions as belonging to another one with a larger resistance bias. Thus the classification boundary of two classes is tilted to the one with larger resistance. The tilt depends on the relative value relationship of resistance bias between the two relationship categories. For example, if we judge the label of a relationship as i , the $\hat{z}_i$ should outperform all of the rest $\{\hat{z}_j, j \neq i\}$ by a necessary classification margin $b_j - b_i$.

4 Experiments

In this section, we evaluate our method on the most popular scene graph generation dataset, Visual Genome (VG). To demonstrate the effectiveness of our method, we compare it with several recent methods, including MOTIFS, VCTree, and BGNN. We also perform comprehensive ablation studies on different components and provide a discussion on different cost-sensitive methods for imbalance learning.

4.1 Dataset and Evaluation Metrics

Dataset We perform extensive experiments on Visual Genome (VG) (?) dataset. Similar to previous work (???), we use the widely adapted subset, VG150, which consists of the most frequent 150 object categories and 50 predicate categories. The original split only has training set (70%) and test set (30%). We follow (?) to sample a 5k validation set for parameter tuning.

Evaluation Metrics For a fair comparison, we follow previous work (???) and evaluate the proposed method on three sub-tasks of scene graph generation as follows.

1. Predicate Classification (**PredCls**). On this task, the input includes not only the image, but the bounding boxes in the image and their labels;
2. Scene Graph Classification (**SGCls**). It only gives the bounding boxes;
3. Scene Graph Detection (**SGDet**). On this task, no additional information other than the raw image will be given.

For each sub-task, previous works use Recall@k (or R@k) to report the performance of scene graph generation (??), which indicates the recall rate of ground-truth relationship triplets (subject-predicate-object) among the top k predictions of each test image. Considering that the above-mentioned metric (R@k) caters to the bias caused by the long-tailed distribution of object relationships, the model can only achieve a high recall rate by predicting the frequent relationships. Therefore, it can not demonstrate the effectiveness of different models on less frequent relationships. To this end, the mean recall rate of each relationships, (mR@k), has been widely used in recent works (????). Therefore, we report the performance using the mean Recall, and the results of Recall are available in the appendix.

4.2 Implementation Details

We implement the proposed method using PyTorch (?). For a fair comparison, we use the object detection model similar to (?), i.e., a pretrained Faster RCNN with ResNeXt-101-FPN as the backbone network. The object detector is fine-tuned on the VG dataset. Then, we froze the object detector and train the rest parts of the SGG model. For the DTrans, the number of object encoder layers is $n_o = 4$ and the number of relationship encoder layers is $n_r = 2$. For the proposed resistance bias, we use $a = 1$ and $\epsilon = 0.001$ if not otherwise stated. For the background relationship, the bias is a constant value $\log \frac{1}{|\mathcal{C}_r|} = \log \frac{1}{50}$, where the $\mathcal{C}_r$ is the set of relationship labels. We perform our experiments using a single NVIDIA V100 GPU. We train the DTrans

Models	Predcls			SGcls			SGdet		
	mR@20	mR@50	mR@100	mR@20	mR@50	mR@100	mR@20	mR@50	mR@100
KERN $\ddagger$ (?)	-	17.7	19.2	-	9.4	10.0	-	6.4	7.3
PCPL $\ddagger$ (?)	-	35.2	37.8	-	18.6	19.6	-	9.5	11.7
GPS-Net $\ddagger$ (?)	17.4	21.3	22.8	10.0	11.8	12.6	6.9	8.7	9.8
BGNN(?)	-	30.4	32.9	-	14.3	16.5	-	10.7	12.6
MOTIFS $\ddagger$ (?)	13.6	17.2	18.6	7.8	9.7	10.3	5.6	7.7	9.2
MOTIFS+TDE(?)	18.5	24.9	28.3	11.1	13.9	15.2	6.6	8.5	9.9
MOTIFS+DLFE(?)	22.1	26.9	28.8	12.8	15.2	15.9	8.6	11.7	13.8
MOTIFS + RTPB(CB)	28.8	35.3	37.7	16.3	19.4	20.6	9.7	13.1	15.5
VCTree $\ddagger$ (?)	13.4	16.8	18.1	8.5	10.8	11.5	5.4	7.4	8.6
VCTree+TDE(?)	17.2	23.3	26.6	8.9	11.8	13.4	6.3	8.6	10.3
VCTree + DLFE(?)	20.8	25.3	27.1	15.8	18.9	20.0	8.6	11.8	13.8
VCTree+ RTPB(CB)	27.3	33.4	35.6	20.6	24.5	25.8	9.6	12.8	15.1
DTrans	15.1	19.3	21.0	9.9	12.1	13.0	6.6	9.0	10.8
DTrans+ RTPB(CB)	30.3	36.2	38.1	19.1	21.8	22.8	12.7	16.5	19.0
DTrans+ RTPB(VB)	25.5	31.1	33.3	17.3	20.1	21.3	11.9	15.7	18.4
DTrans+ RTPB(PB)	17.4	21.6	23.1	11.9	14.0	14.7	7.7	10.1	11.9
DTrans+ RTPB(EB)	22.7	26.7	28.4	15.2	17.4	18.2	11.0	14.1	16.1

Table 1: The performance on VG (?) under graph constraints setting. † indicates the results reproduced using the code of (?). ‡ models are with VGG16 backbone (?), while others are with ResNeXt-101-FPN backbone (?). (CB), (VB), (PB), and (EB) indicate the count resistance bias, the valid resistance bias, the pair resistance bias, and the estimated resistance bias, respectively.

model for 18000 iterations with batch size 16. Specifically, it takes around 24 hours for the SGDet task, and less than 12 hours for the PredCls/SGCls task. For the SGDet task, we use all pairs of objects for training instead of those pair of objects with overlap. For other experimental settings, we always keep the same with the previous work (?).

4.3 Comparison with Recent Methods

To demonstrate the effectiveness of the proposed method, we apply RTPB on several representative methods for scene graph generation, including MOTIFS, VCTree, and our DTrans baseline model. As shown in Table 1, we find that RTPB can achieve consistent improvements in the mean recall metric for all of MOTIFS, VCTree, and our DTrans. Models equipped with our RTPB achieve new state-of-the-art and out-perform previous methods by a clear margin. Take SGDet as an example, as shown in Figure 4, the recall rate on most relationships has been significantly improved (DTrans w/o or w/ RTPB). Specifically, RTPB may also lead to slight performance degradation on the frequent relationships without over-fitting on the head categories. Therefore, we see a clear improvement when using the mean recall rate as the evaluation metric, which demonstrates RTPB can build a better-balanced model for unbiased SGG.

Among four instance of resistance bias, CB performs the best on mR because it simply calculates the relationship distribution based on the whole dataset. VB/PB/EB are proposed based on more critical conditions. Although we believe that adding more critical conditions would lead to more practical prior bias, currently, limited data are not sufficient to obtain proper empirical bias under the given conditions.

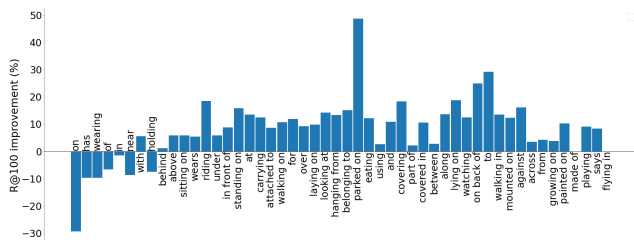


Figure 4: The improvements of Recall@100 on the SGDet task using RTPB(CB). The number of relationships decreases from left to right. It shows that RTPB achieves higher recall rate on most relationships with a sacrifice on only a small number of frequent relationships.

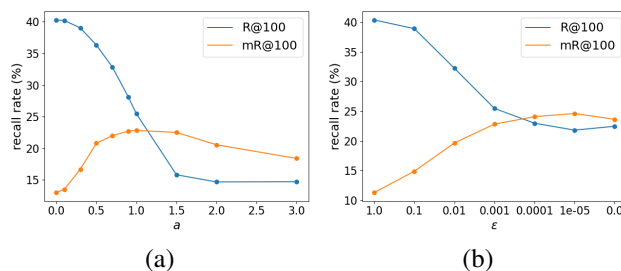


Figure 5: (a) the influence of different a . (b) the influence of different ϵ .

4.4 Ablation Studies

In this subsection, we evaluate the influence of important hyper-parameters (a and ϵ) and an extended version of resistance bias, i.e., soft resistance bias. Besides, we compare our RTPB with several conventional imbalance learning meth-

ods on the SGG task.

Hyper-parameters for Resistance Bias Two important hyper-parameters in RTPB, a and ϵ are used to control the distribution of the resistance bias. Specifically, $a > 0$ is used to adjust the divergence of the bias, while a small $\epsilon \in [0, 1]$ is used to control the maximum relative difference. We perform experiments on the SGCIs task to evaluate the influence of a and ϵ in the RTPB. As shown in Figure 5(a), when increasing a , the overall recall rate decreases and the mean recall rate increases; when $a > 1.0$, the mean recall rate also decreases. Therefore, we use $a = 1.0$ in our experiments. As shown in Figure 5(b), we find that $\epsilon = 1e-3$ achieves a good trade-off between the overall recall rate and the mean recall rate. Furthermore, we find out that when either $a = 0$ or $\epsilon \leq 1$, the performance is very close to the baseline method because the resistance biases for different relationships are close to each other.

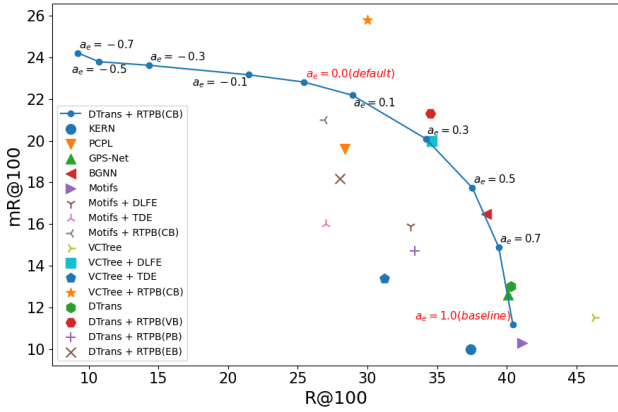


Figure 6: The result of inference with different a_e .

Soft Resistance Bias To better understand how RTPB addresses the trade-off between head and tail relationships, we introduce a soft version of resistance bias during inference phase as follows.

$$b_i^e = -\log\left(\frac{w_i^{a_e}}{\sum_{j \in \mathcal{C}_r} w_j^{a_e}} + \epsilon\right), \quad (14)$$

where $a_e \leq a$, and b_i^e is the smooth version of resistance bias. By applying a smoothed resistance bias in the inference phase, we can evaluate different impacts of RTPB on the trained model. Following the analysis of the resistance bias for adjusting the classification boundary, a smooth resistance bias reduces the margin areas of head categories from $\text{alog } \frac{w_i}{w_j}$ to $(a - a_e)\text{log } \frac{w_i}{w_j}$. Specifically, 1) when $a_e = 0$, the result is same as the original one; and 2) when $a_e = a = 1$, the result is close to the baseline without RTPB, i.e., Motifs (?) and DTrans. The experimental results of different a_e are shown in Figure 6. The model used for evaluation is trained with CB and $a = 1$. As the a_e decreases, the classification margin of head categories keeps becoming larger. Therefore, the model performs worse on the few head categories but performs better on the rest tail categories. For

Loss	mR@20	mR@50	mR@100
Baseline	9.9	12.2	13.0
$\mathcal{L}_{Reweight}$	9.9	14.3	16.8
$\mathcal{L}_{ClsBal}$	12.8	15.5	17.0
$\mathcal{L}_{focal}$	9.1	11.3	12.1
$\mathcal{L}_{LDAM}$	8.4	10.3	10.8
RTPB(CB)	19.1	21.8	22.8

Table 2: Comparison with other cost-sensitive methods.

the evaluation result, this means a consistent improvement in mean recall metric as shown in Figure 6. Besides, the model trained with RTPB can achieve similar performance as many current SGG methods with different a_e . Thus RTPB can be regarded as a general baseline for different unbiased SGG methods.

4.5 Other Cost-sensitive Methods

As shown in Table 2, we compare our method with the following cost-sensitive methods on the SGCIs task, using our DTrans as the backbone. 1) **Re-weighting Loss**: using the fraction of the count of each class as the weight of loss. 2) **Class Balanced Re-weighting Loss (?)**: using the effective number of samples to re-balance the loss. 3) **Focal Loss (?)**: using the focal weight to adjust the losses for well-learned samples and hard samples. 4) **Label Distribution-Aware Margin Loss (?)**: using the prior margin for each class to balance the model.

We evaluate these methods on the SGCIs task with our DTrans. As shown in Table 2, our method outperforms the rest of the methods by a clear margin in the mean recall.

5 Conclusion

In this paper, we propose a novel resistance training strategy using prior bias (RTPB) for unbiased SGG. Experimental results on the popular VG dataset demonstrate that our RTPB achieves a better head-tail trade-off for the SGG task than existing counterparts. We also devise a novel transformer-based contextual encoding structure to encode global information for visual objects and relationships. We obtain significant improvements over recent state-of-the-art approaches, and thus set a new baseline for unbiased SGG.

Supplementary Material

Visualization of the loss

The softmax CE with our resistance bias is calculated as follows:

$$\begin{aligned}
 \hat{\mathcal{L}}_i &= -\log(\hat{p}_i) = -\log \frac{e^{z_i - b_i}}{\sum_{j \in \mathcal{C}_r} e^{z_j - b_j}} \\
 &= -\log \frac{e^{z_i}}{\sum_{k \in \mathcal{C}_r} e^{z_k}} \frac{e^{-b_i} \sum_{k \in \mathcal{C}_r} e^{z_k}}{\sum_{j \in \mathcal{C}_r} e^{z_j - b_j}} \\
 &= -\log \frac{e^{z_i}}{\sum_{k \in \mathcal{C}_r} e^{z_k}} + b_i + \log \sum_{j \in \mathcal{C}_r} \frac{e^{-b_j} e^{z_j}}{\sum_{k \in \mathcal{C}_r} e^{z_k}} \\
 &= -\log p_i + b_i + \log \sum_{j \in \mathcal{C}_r} e^{-b_j} p_j \\
 &= \mathcal{L}_i + \theta_i,
 \end{aligned} \tag{15}$$

where $\mathcal{L}_i$ is the vanilla CE, and $\theta_i = b_i + \log \sum_{j \in \mathcal{C}_r} e^{-b_j} p_j$ is the extra weight of loss value, which is correlated to the resistance bias value for each type. Specifically, the extra weight θ_i increase the loss value when the resistance bias is large.

Take binary classification as an example, we can visualize the loss value for the two classes with specific resistance bias values. The loss values for several pair of resistance bias values are shown in Figure 7.

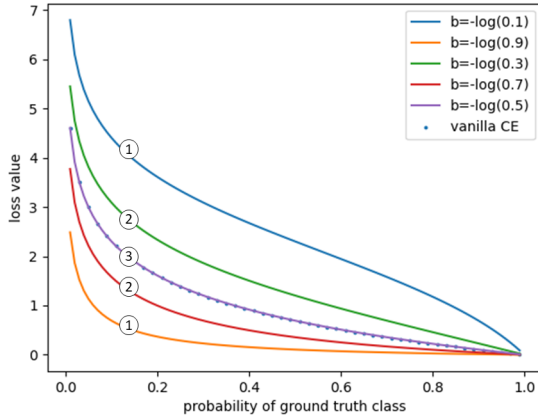


Figure 7: The loss value for cross-entropy with our RTPB. This figure shows three pairs of loss with different resistance bias. In the pair ①, the resistance bias is set to be $-\log(0.9)$ and $-\log(0.1)$. $-\log(0.7)$ and $-\log(0.1)$ is the bias value for the second pair, ②. In the last pair, ③, two classes take the same resistance bias value, and the loss value is equivalent to the vanilla CE (?).

Experiment Results

Table 3 shows the Recall results and mean Recall results on the VG dataset (?). To clearly compare all methods, we visualize the results on the PredCls task, SGCLs task, and SGDet

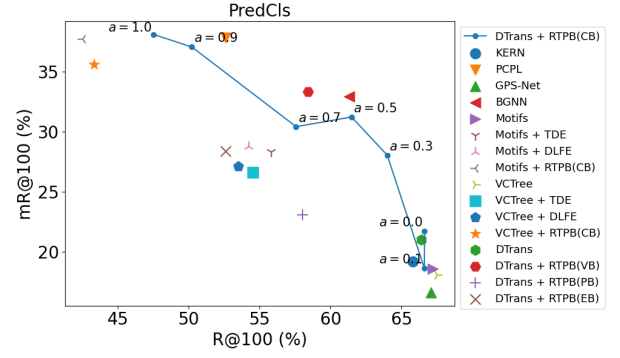


Figure 8: The distribution of R@100 and mR@100 in PredCls task.

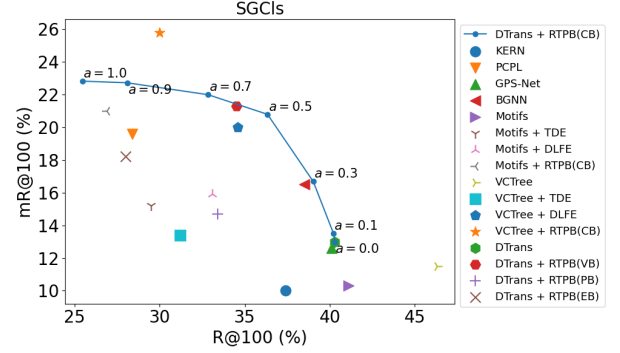


Figure 9: The distribution of R@100 and mR@100 in SGCLs task.

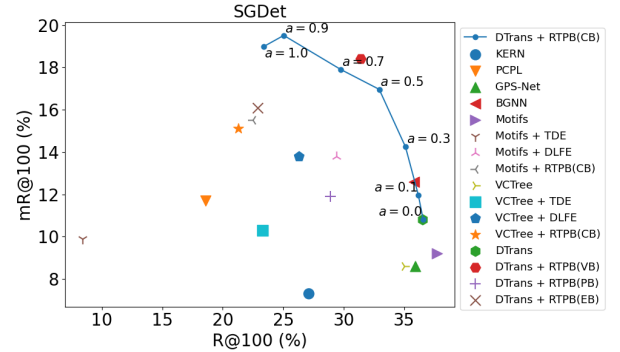


Figure 10: The distribution of R@100 and mR@100 in SGDet task.

task in Figure 8, 9 and 10, respectively. As shown in Figure 8, 9 and 10, models trained with RTPB achieve higher mean recall than previous state-of-the-art methods. Besides, the model trained with different a for RTPB provide examples for head-tail trade-off in SGG.

Moreover, in addition to Recall and mean Recall result with graph constraint, we also report the results without graph constraint of our methods in Table 4. As we can observe, RTPB also significantly improves the mean Re-

Models	Predcls		SGcls		SGdet	
	mR@50/100	R@50/100	mR@50/100	R@50/100	mR@50/100	R@50/100
KERN $\ddagger$ (?)	17.7/19.2	67.6/65.8	9.4/10.0	36.7/37.4	6.4/7.3	29.8/27.1
PCPL $\ddagger$ (?)	35.2/37.8	50.8/52.6	18.6/19.6	27.6/28.4	9.5/11.7	14.6/18.6
GPS-Net $\ddagger$ (?)	21.3/22.8	66.9/68.8	11.8/12.6	39.2/40.1	8.7/9.8	28.4/31.7
BGNN(?)	30.4/32.9	59.2/61.3	14.3/16.5	37.4/38.5	10.7/12.6	31.0/35.8
MOTIFS $\dagger$ (?)	17.2/18.6	65.4/67.2	9.7/10.3	40.3/41.1	7.7/9.2	33.0/37.7
MOTIFS+TDE(?)	24.9/28.3	50.8/55.8	13.9/15.2	27.2/29.5	8.5/9.9	7.4/8.4
MOTIFS+DLFE(?)	26.9/28.8	52.5/54.2	15.2/15.9	32.3/33.1	11.7/13.8	25.4/29.4
MOTIFS + RTPB(CB)	35.3/37.7	40.4/42.5	20.0/21.0	26.0/26.9	13.1/15.5	19.0/22.5
VCTree $\dagger$ (?)	16.8/18.1	65.8/67.5	10.8/11.5	45.4/46.3	7.4/8.6	31.0/35.0
VCTree+TDE(?)	23.3/26.6	49.9/54.5	11.8/13.4	28.8/31.2	8.6/10.3	19.6/23.3
VCTree + DLFE(?)	25.3/27.1	51.8/53.5	18.9/20.0	33.5/34.6	11.8/13.8	22.7/26.3
VCTree+RTPB(CB)	33.4/35.6	41.2/43.3	24.5/25.8	28.7/30.0	12.8/15.1	18.1/21.3
DTrans	19.3/21.0	64.4/66.4	12.1/13.0	39.4/40.3	9.0/10.8	32.1/36.5
DTrans+RTPB(CB)	36.2/38.1	45.6/47.5	21.8/22.8	24.5/25.5	16.5/19.0	19.7/23.4
DTrans+RTPB(VB)	31.1/33.3	56.5/58.4	20.1/21.3	33.6/34.5	15.7/18.4	27.3/31.4
DTrans+RTPB(PB)	21.6/23.1	56.2/58.0	14.0/14.7	32.6/33.4	10.1/11.9	24.6/28.9
DTrans+RTPB(EB)	26.7/28.4	50.6/52.6	17.4/18.2	27.0/28.0	14.1/16.1	19.1/22.9

Table 3: The performance on VG (?) under graph constraints setting. $\dagger$ indicates the results reproduced using the code of (?). $\ddagger$ models are with VGG16 backbone (?), while others are with ResNeXt-101-FPN backbone (?). (CB), (VB), (PB) and (EB) indicate the count resistance bias, the valid resistance bias, the pair resistance bias, and the estimated resistance bias, respectively.

call of corresponding methods. Besides, DTrans+RTPB(CB) performs the best on nearly all mean Recall. The Recall is dropped by using RTPB. This is because the less frequently seen relationships have been highlighted, where most ground truths are frequently seen. Therefore, it is commonly seen in unbiased SGG strategies that an increase of mean Recall would be followed with a decrease of Recall (?).

Cost-sensitive Methods

We compare our RTPB with the following cost-sensitive methods.

1) **Re-weighting Loss**: This strategy uses the fraction of the count of each class to weigh the loss value as follows,

$$\mathcal{L}_{Rweight}(i) = \frac{1}{n_i} \mathcal{L}_i, \quad (16)$$

where n_i is the number of all samples of i -th class in training set. For SGG, n_i is set to be the number of relationship i .

2) **Class Balanced Re-weighting Loss** (?): This method uses the inversed effective number of samples to re-balance the loss. The loss is calculated as

$$\mathcal{L}_{ClsBal}(i) = \frac{1}{n_i^{eff}} \mathcal{L}_i = \frac{1-\beta}{1-\beta^{n_i}} \mathcal{L}_i, \quad (17)$$

where $n_i^{eff} = \frac{1-\beta}{1-\beta^{n_i}}$ is the effective number for class i , n_i is the number of sample of class i . Following the setting of (?), we set the hyper-parameter $\beta = 0.999$.

3) **Focal Loss** (?): This method uses the focal weight to adjust the loss value for well-learned samples and focuses on the hard ones.

$$\mathcal{L}_{focal}(p_i) = -\alpha(1-p_i)^\gamma \log(p_i), \quad (18)$$

where p_i is the classification probability for class i . We followed the hyper-parameters ($\gamma = 2.0, \alpha = 0.25$).

4) **Label Distribution-Aware Margin Loss** (?): This method uses the margin for each class to balance the model. The loss value for class i is

$$\mathcal{L}_{LDAM}(i) = -\log \frac{e^{z_i - \Delta_i}}{e^{z_i - \Delta_i} + \sum_{j \neq i} e^{z_j}} \quad (19)$$

where $\Delta_i = \frac{C}{n_i^{1/4}}$

where C is the constant for the margin size and we set it as 0.5 same as (?), and n_i is the number of sample of the corresponding class i in the training set. This class-related loss can be regarded as a special form of our method if we set the bias corresponding to the ground truth as follow

$$b_i = \begin{cases} \Delta_i, & i = gt \\ 0, & i \neq gt \end{cases}, \quad (20)$$

where gt is the groundtruth of relationship. We evaluate methods mentioned above on the VG data set(?) with our DTrans, and find out that our RTPB outperforms the rest of the methods by a clear margin in the mean recall metric. Full results are shown in the Table 5.

Qualitative Study

We visualized several SGCLs examples generated by the DTrans and the DTrans with RTPB(CB) in Figure 11. As shown in the figure, the model without RTPB prefers generalized descriptions like *on* and *has*, which provide less valuable information. This is because these generalized relationships or the frequently seen relationships dominate the training set. However, the model trained with RTPB tends to replace these rough-grained relationships with fine-grained and more informative ones like *parked on*, *mounted on*, and *painted on*, which constitutes more meaningful scene

Models	Predcls		SGcls		SGdet	
	mR@50/100	R@50/100	mR@50/100	R@50/100	mR@50/100	R@50/100
MOTIFS†(?)	33.8/46.0	81.7/88.9	19.9/26.8	50.3/54.0	13.2/17.9	37.2/44.1
MOTIFS+TDE§ (?)	29.0/38.2	-	16.1/21.1	-	11.2/14.9	-
MOTIFS+DLFE§ (?)	30.0/45.8	-	25.6/32.0	-	18.1/23.0	-
MOTIFS + RTPB(CB)	48.0/58.9	62.8/74.9	27.2/33.3	39.6/46.5	17.5/22.5	25.7/32.6
VCTree†(?)	34.6/46.9	82.5/89.5	22.2/30.7	56.3/60.6	12.6/16.8	33.6/39.9
VCTree+TDE§ (?)	32.4/41.5	-	19.1/25.5	-	11.5/15.2	-
VCTree + DLFE§ (?)	44.6/56.8	-	31.4/38.8	-	17.5/22.5	-
VCTree+RTPB(CB)	46.4/57.8	62.6/74.9	34.0/41.1	45.1/53.7	17.3/21.9	24.5/31.4
DTrans	36.6/48.8	80.5/87.9	23.2/29.7	49.5/53.3	14.4/19.6	36.3/43.0
DTrans+RTPB(CB)	52.6/63.5	70.0/80.7	31.6/37.1	42.0/48.5	22.4/27.5	27.5/35.3
DTrans+RTPB(VB)	47.5/59.7	76.5/85.3	30.6/36.9	46.6/51.4	21.3/27.2	32.5/39.8
DTrans+RTPB(PB)	38.6/50.4	79.2/87.7	25.4/31.9	48.4/52.8	16.6/21.7	33.4/41.1
DTrans+RTPB(EB)	42.5/52.6	76.4/85.9	27.5/33.0	46.1/51.6	19.9/24.7	30.4/38.2

Table 4: The performance on VG (?) without graph constraint. †, ‡, (CB), (VB), (PB) and (EB) are with the same meaning as in Table 3. § denote the results are reported by (?)

Models	Predcls		SGcls		SGdet	
	mR@50/100	R@50/100	mR@50/100	R@50/100	mR@50/100	R@50/100
Baseline	19.3/21.0	64.4/66.4	12.1/13.0	39.4/40.3	9.0/10.8	32.1/36.5
$\mathcal{L}_{Reweight}$	35.1/ 38.8	26.8/30.6	14.3/16.8	13.6/15.6	2.0/2.6	4.0/5.1
$\mathcal{L}_{ClsBal}$	19.9/21.9	64.5/66.3	15.5/17.0	38.8/39.7	10.7/12.8	31.8/36.2
$\mathcal{L}_{focal}$	19.9/21.7	63.2/65.6	11.3/12.1	38.6/39.8	8.3/10.0	29.6/33.9
$\mathcal{L}_{LDAM}$	15.3/16.6	65.1/66.9	10.3/10.8	39.6/40.4	7.0/8.6	32.3/37.1
RTPB(CB)	36.2/38.1	45.6/47.5	21.8/22.8	24.5/25.5	16.5/19.0	19.7/23.4

Table 5: Comparison of Recall rate and mean Recall rate (with graph constraint) with other cost-sensitive methods.

Models	Predcls		SGcls		SGdet	
	mR@50/100	R@50/100	mR@50/100	R@50/100	mR@50/100	R@50/100
Baseline	19.3/21.0	64.4/66.4	12.1/13.0	39.4/40.3	9.0/10.8	32.1/36.5
$\mathcal{L}_{Reweight}$	41.2/50.0	40.4/53.0	16.4/21.1	20.3/26.7	2.7/3.7	5.4/7.0
$\mathcal{L}_{ClsBal}$	38.4/51.5	80.3/87.9	26.9/34.7	49.0/52.9	16.9/22.3	35.9/42.7
$\mathcal{L}_{focal}$	27.0/36.9	76.6/84.8	18.4/24.2	46.9/51.4	12.4/16.8	33.2/39.5
$\mathcal{L}_{LDAM}$	33.6/44.9	80.2/87.9	22.6/28.9	49.0/52.9	13.0/17.9	34.8/42.1
RTPB(CB)	52.6/63.5	70.0/80.7	31.6/37.1	42.0/48.5	22.4/27.5	27.5/35.3

Table 6: Comparison of Recall rate and mean Recall rate (without graph constraint) with other cost-sensitive methods.

graphs. The differences between the results from the two models demonstrate the necessity of unbiased scene graph generation and the effectiveness of our proposed RTPB.

Acknowledgements

This work was supported in part by the National Natural Science Foundation of China under Grants 61822113, 41871243, 62002090, and the Science and Technology Major Project of Hubei Province (Next-Generation AI Technologies) under Grant 2019AEA170. Dr. Baosheng Yu is supported by ARC project FL-170100117. Dr. Liu Liu is supported by ARC project DP-180103424.

Quidem maxime placeat rem totam vitae iusto maiores earum eos modi, modi esse deserunt provident enim similique?Exercitationem architecto magni eligendi, tempora temporibus mollitia tempore perferendis ex hic quam non, hic quam nam consequuntur suscipit facere obcaecati ducimus commodi ab illum ullam, quisquam quia prae-

sentium eaque voluptate.Magnam unde obcaecati exercitationem ad ducimus cumque reiciendis sunt nesciunt quaeat, provident rerum cumque?Nam fugit quibusdam quisquam inventore illum nobis saepe molestiae ratione, voluptatum quidem accusamus voluptates provident dignissimos autem, consequatur aliquam iure id recusandae repellat reiciendis dolore provident ratione, dolorum doloreque voluptatibus autem, labore excepturi praesentium ut repellendus cum.Animi id molestiae ratione fugit assumenda ab asperiores perspicatis, laudantium obcaecati beatae in saepe, fugiat aperiam at exercitationem dolorem?Maiores distinctio eaque itaque molestias delectus debitis dicta incidunt officiis maxime labore, sunt quod dolores expedita eius quam doloribus dicta blanditiis culpa?Animi laborum beatae voluptate reprehenderit, laudantium possumus perferendis earum ex laborum ut aperiam magnam iste explicabo, modi dicta aut expedita quibusdam facilis consectetur sequi, sequi ducimus nulla voluptate alias, nam ipsam saepe fa-

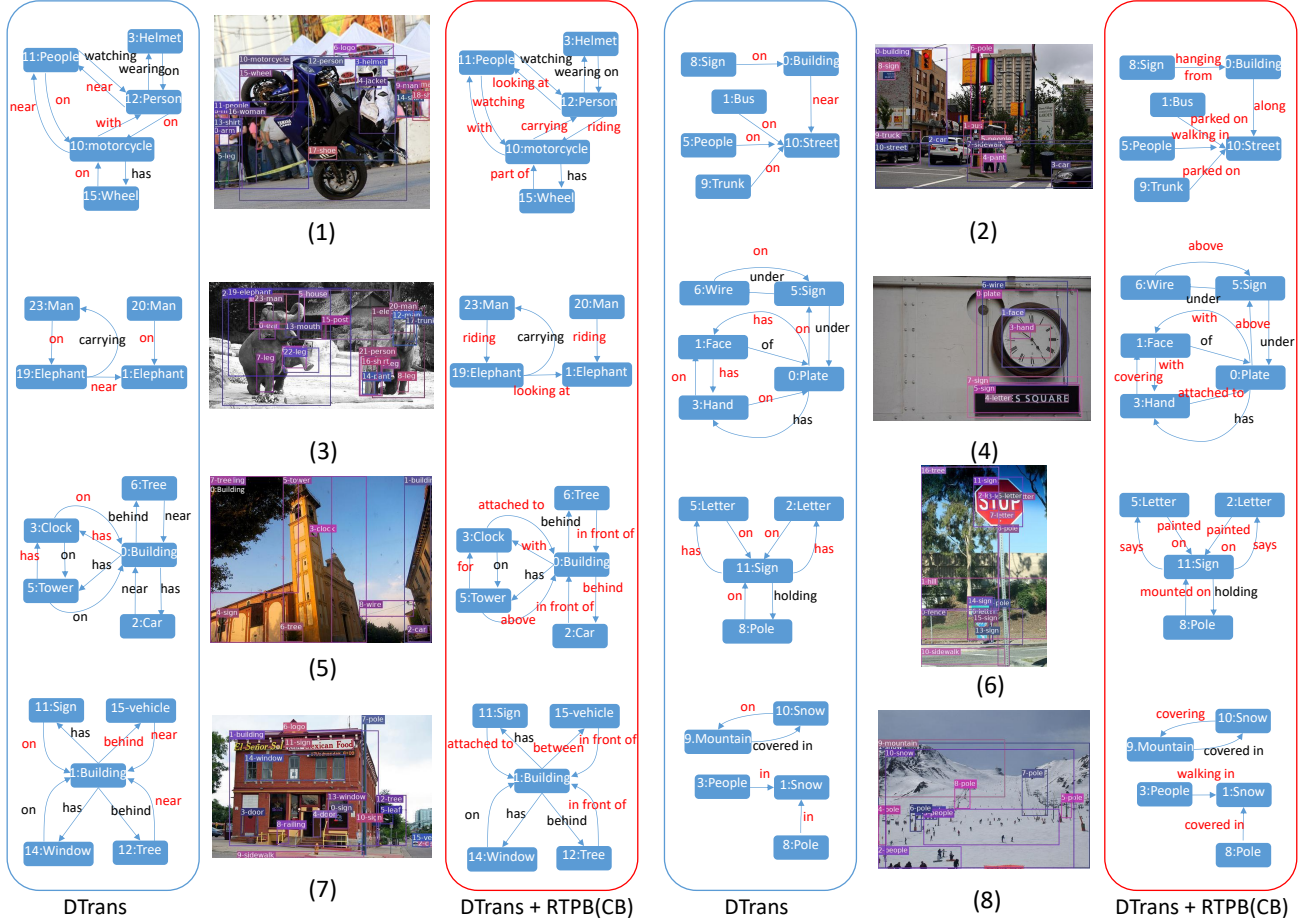


Figure 11: This figure shows several sample of SGCs. For each sample, we present the groundtruth of object detection in the image and two scene graphs generated by different methods. The scene graphs in the blue boxes are produced by our DTrans without RTPB, and the ones in the red boxes are produced by the DTrans trained with RTPB(CB). We highlight the differences between each pair of scene graphs. Because of the space limitation, we exclude part of trivial detected objects in generated scene graphs.

cilis optio minus aspernatur amet. Recusandae hic ut dolores officia nam, quasi ullam quod accusamus reiciendis nihil maiores officiis omnis vitae odio, molestiae rem tempore blanditiis ab aliquam, fugiat facere explicabo cupiditate, soluta praesentium autem similique consequuntur accusantium. Suscipit voluptatem reprehenderit enim illum eum dolorem animi, dolorem libero iure unde maiores laboriosam. Quidem voluptatum minus illo saepe provident similique dicta iste pariatur, iusto corrupti accusamus dolor non, ab cumque rerum mollitia delectus laboriosam amet eius, incidunt dolorem eaque eum facere earum vero quasi. Hic quo consectetur, vitae quibusdam dolores dolore veniam eveniet labore in velit consectetur laborum reiciendis. Perferendis eius tenetur quas debitis iure est optio temporibus placeat, natus illum pariatur nobis, amet velit rerum impedit quidem. Cupiditate quidem est iure unde expedita facere nihil

vero, nisi odit quis unde hic necessitatibus cupiditate nam totam neque nulla sapiente, voluptatibus vitae quae numquam voluptatum in quam, a veritatis nobis in eum? Eligendi obcaecati magnam distinctio voluptate at nulla vero nesciunt vitae qui accusamus, unde voluptate quibusdam, tempora cupiditate pariatur sapiente quas tenetur saepe incidunt, dolore et hic earum omnis molestias unde a expedita enim, quibusdam non ab? Dolorem iusto voluptatum enim, quo saepe modi beatae, ab molestiae non odio quae quisquam voluptatum, nemo veniam neque ipsam. Deserunt minima autem quis, repudiandae obcaecati porro assumenda ratione est sit ad consectetur quos, excepturi ducimus saepe eveniet ad enim aliquid quo placeat optio rerum fugit, totam nulla necessitatibus, facere a sit. Aperiam ex laboriosam, explicabo hic labore ad nobis excepturi. Magnam ducimus aliquam deleniti vero minima nemo molestias in, dolores

numquam iusto dolor assumenda corrupti nesciunt incidunt
excepturi reiciendis fugit ad.Minima beatae ut tenetur nihil
aliquid, sint temporibus assumenda dicta,